

Project Documentation – Tableau & Flask Stack

Project Documentation format

1. Introduction

- Project Title: Heritage Treasures: An In-Depth Analysis of UNESCO World Heritage Sites in Tableau
- Team Members: Devendra Reddy (Team Lead), DHARA MAHITHA (Member)

2. Project Overview

- Purpose: To visualize the distribution, danger status, and regional inscription trends of UNESCO World Heritage Sites so conservation analysts, policy makers, and tourism boards can make informed preservation decisions.
- Features: Country-wise treemap, In-Danger vs Not-in-Danger pie chart, regional inscription trend line, an interactive filterable dashboard, a guided multi-scene story, and a Flask-embedded public view.

3. Architecture

- Frontend: The 'frontend' is the Tableau-authored dashboard and story (treemap, pie chart, line chart, filters) rendered through the Tableau JavaScript Embedding API inside HTML/CSS/Bootstrap pages served by Flask.
- Backend: Python (Flask) serves the web routes/templates that embed the Tableau views and handles any request parameters.
- Database: No relational/NoSQL database is used — the UNESCO World Heritage Sites (2019) CSV is the single source of data, cleaned with Pandas and loaded as a Tableau extract (.hyper).

4. Setup Instructions

- Prerequisites: Python 3.x, Flask, Pandas, Tableau Desktop (for authoring) and a Tableau Public/Server account (for publishing).
- Installation: Clone the repository, run 'pip install -r requirements.txt --break-system-packages', place the cleaned dataset/workbook link in the config, then set the TABLEAU_EMBED_URL environment variable to the published dashboard link.

5. Folder Structure

- Client: templates/ (HTML pages with the Tableau embed code) and static/ (CSS, JS, images).
- Server: app.py (Flask routes), data/ (raw and cleaned CSV files), notebooks/ (Pandas cleaning scripts), and workbook/ (the Tableau .twbx file).

6. Running the Application

- Provide commands to start the application locally.
 - o Data prep: 'python clean_data.py' in the data directory.
 - o Web app: 'python app.py' in the project root, then open `http://127.0.0.1:5000`

7. API Documentation

- The Flask app exposes simple page routes rather than a REST API: '/' (dashboard view) and '/story' (story view), each rendering a template that embeds the corresponding published Tableau view.
- No JSON API is required since Tableau serves and renders the visual

data directly; if extended, a '/api/summary' endpoint could return

aggregate counts as JSON for future features.

- Authentication is not required for this project — the dashboard/story is publicly viewable. Publishing access to Tableau Server/Public itself is controlled by Tableau's own account permissions.

9. User Interface

- See the Results section of the Final Report for dashboard and story screenshots showing the treemap, pie chart, trend line, filters, and story navigation.

10. Testing

- Manual User Acceptance Testing was performed (6 test cases, all Pass – see the User Acceptance Testing FSD document) covering chart accuracy, filter interaction, story navigation, and the Flask embed.

11. Screenshots or Demo

- See the Final Report / Project Demonstration folder for dashboard and story

screenshots and the demo link once published by the team lead.

- Known Issues: first load of the embedded dashboard can show a brief loading spinner;

story captions may overflow on very small mobile widths.

- Future Enhancements: add a live/incremental data connection instead of a static extract, include a geographic map view, and add a simple '/api/summary' endpoint for programmatic access.